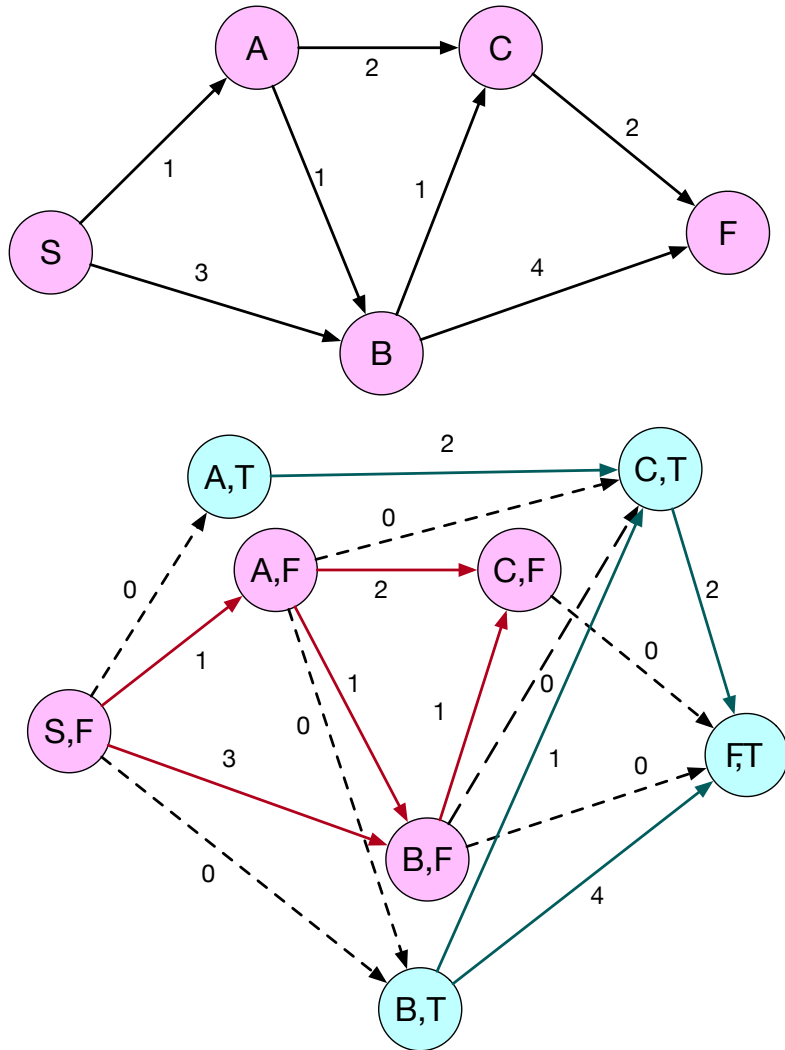


CS 3510 Algorithms: Intro to Dynamic Programming

Aaron Hillegass
Georgia Tech

Problem 4



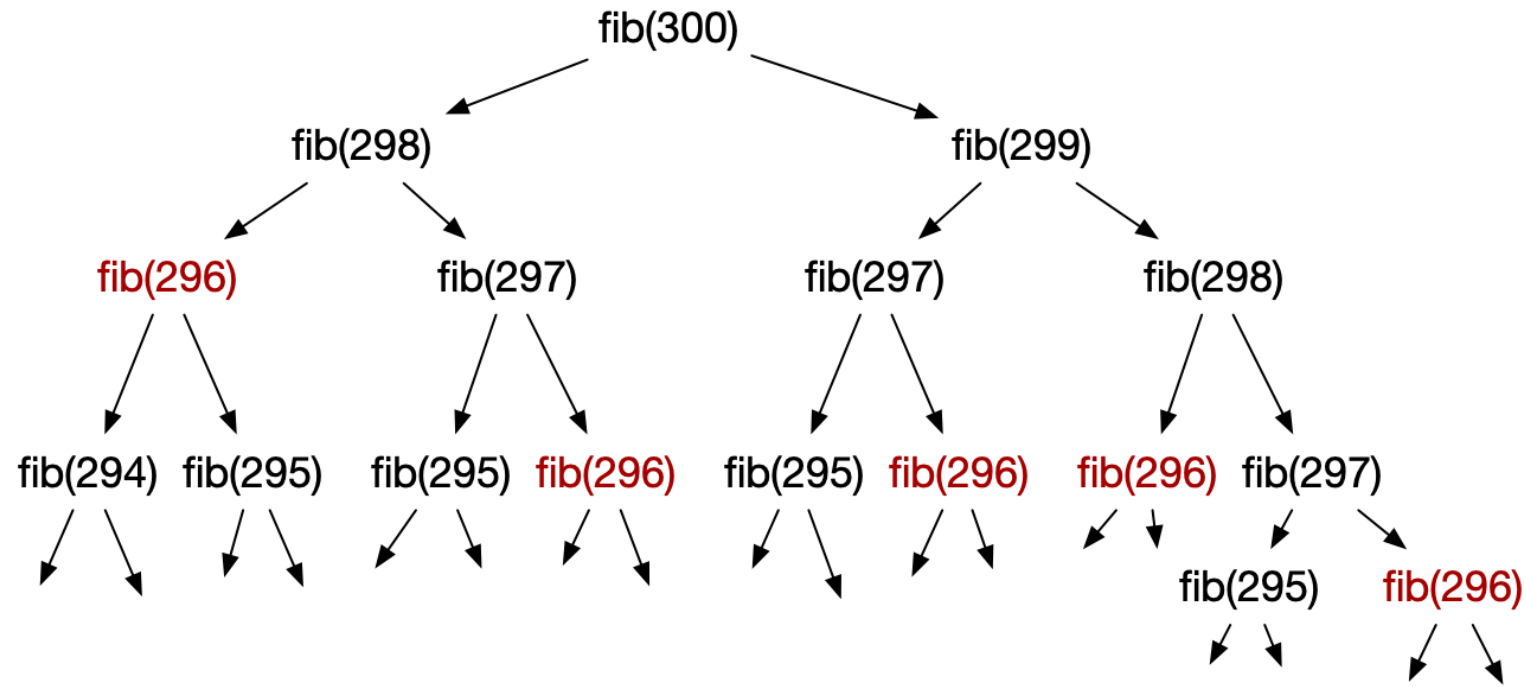
```

1 define search( $G, v_s, v_f$ ):
2   prev := new map, q := new priority queue
3   q.insert( $((v_s, \text{False}), 0)$ )
4   while ( $v, \text{has\_used}$ ),  $d := q.\text{pop\_head}()$ :
5     if  $v = v_f$  and has_used = True:
6       | return backtrack(prev, ( $v_f, \text{True}$ ))
7     forall ( $(v, u), c \in G.E$ )
8       if has_used:
9         | if q.decrease_or_insert( $((u, \text{True}), d + c)$ ):
10          | prev[ $((u, \text{True}))$ ] := ( $v, \text{True}$ )
11       else:
12         | if q.decrease_or_insert( $((u, \text{False}), d)$ ):
13          | prev[ $((u, \text{False}))$ ] := ( $v, \text{False}$ )
14         | if q.decrease_or_insert( $((u, \text{False}), d + c)$ ):
15          | prev[ $((u, \text{False}))$ ] := ( $v, \text{False}$ )

```

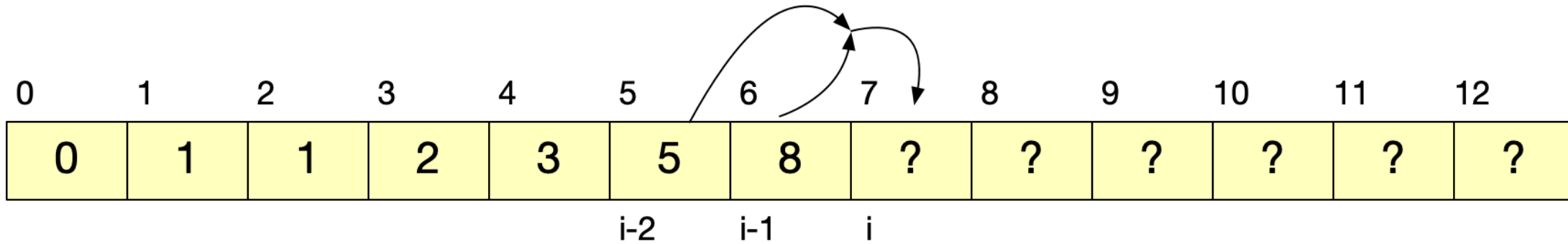
Bad Recursion

3/56



Memoization: Calculate `fib(296)` *once*, store it, and never calculate it again!

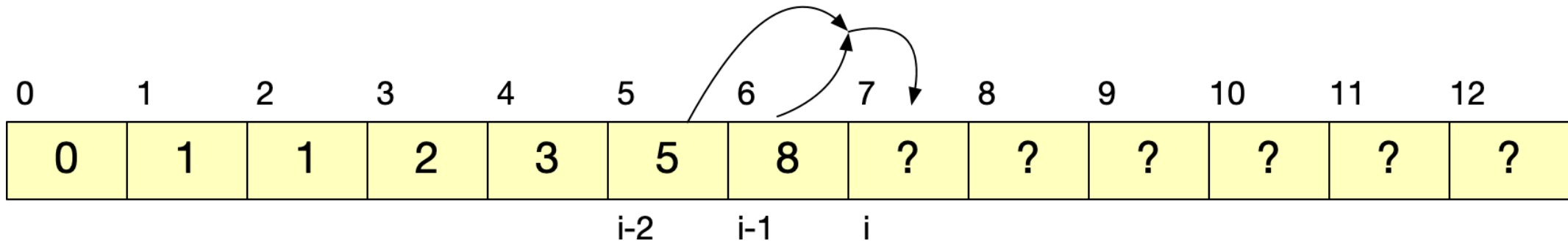
Dynamic Programming fib(12)



```
1 define fib( $n$ ):  
2    $\text{dp} := \text{new array } [0 \dots n]$   
3    $\text{dp}[0] := 0$   
4    $\text{dp}[1] := 1$   
5   for  $i$  in  $2 \dots n$   
6      $\text{dp}[i] := \text{dp}[i - 2] + \text{dp}[i - 1]$   
7   return  $\text{dp}[n]$ 
```

Dynamic Programming fib(n)

- Complexity: $O(n)$
- Space: $O(n)$



What is “Dynamic Programming”?

- Uses memoization to prevent redundant computation
- Works from base cases up (inductively) instead of from the top down (recursively)

Complexity and space tend to be the same in dynamic programming.

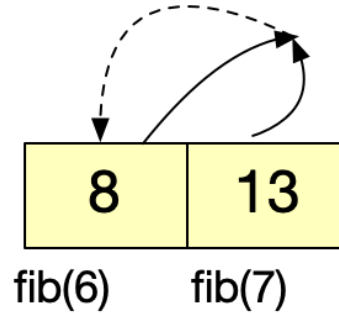
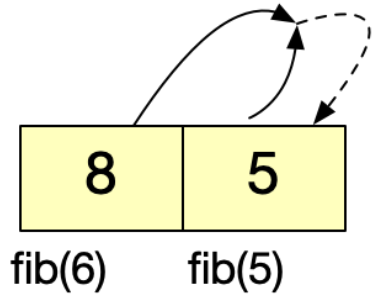
(Assuming you aren't reusing or freeing memory as you go.)

(But you probably should, if you can.)

Tidy Memoization

Calculate `fib(296)` once, and store it *until you don't need it any more!*

Tidy Dynamic Programming `fib(n)`

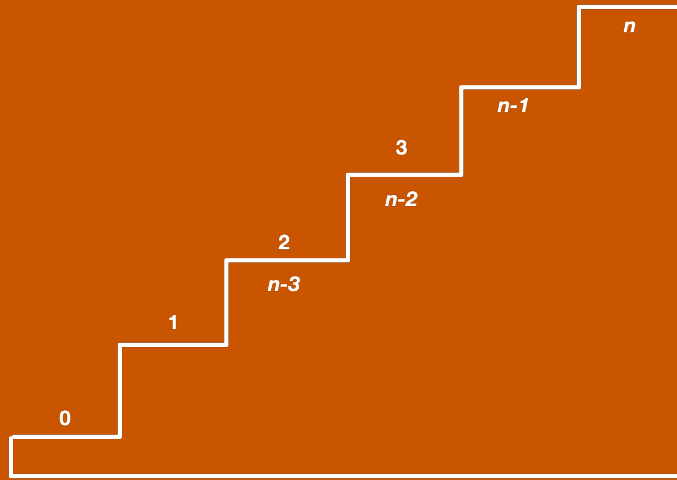


```

1 define fib(n):
2   dp := new array [0...1]
3   dp[0] := 0
4   dp[1] := 1
5   for i in 2...n
6     | dp[i mod 2] := dp[0] + dp[1]
7   return dp[n mod 2]

```

You can hop up 1, 2, or 3 steps. How many different ways are there to get to step n ?

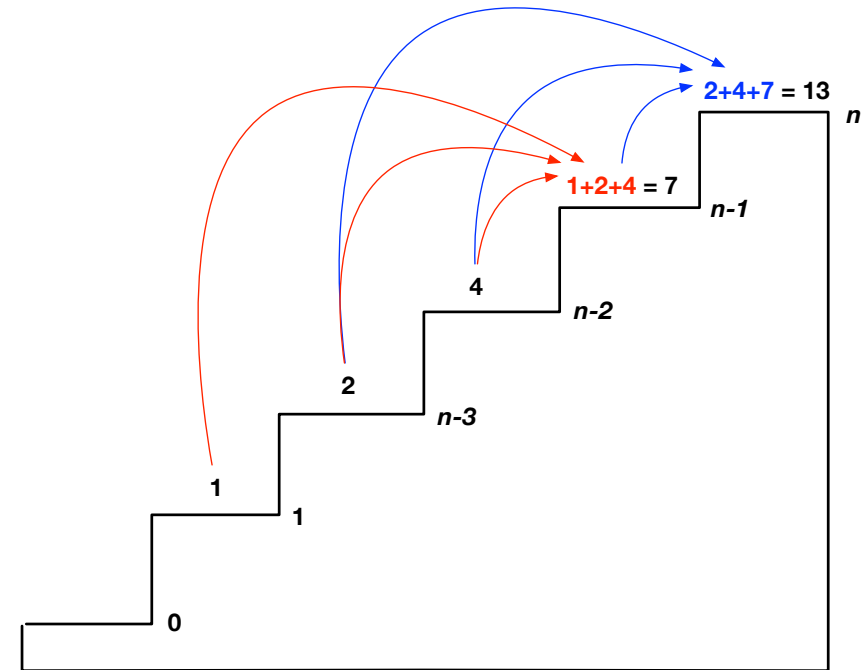


Take 1,2 or 3 steps

```

1 define routes( $n$ ):
2    $dp := \text{array}[1..n]$ 
3    $dp[1] := 1$ 
4    $dp[2] := 2$ 
5    $dp[3] := 4$ 
6   for  $i$  in  $[4..n]$ :
7      $dp[i] := dp[i - 3] + dp[i - 2] + dp[i - 1]$ 
8   return  $dp[n]$ 

```

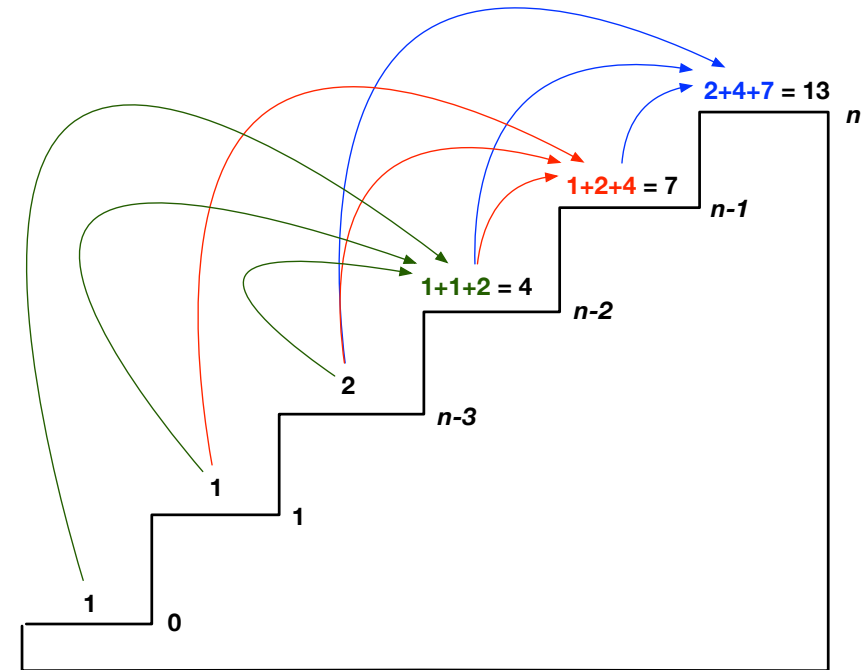


How could you save space?

What would be a good data structure
for this?

Base cases can be strange

```
1 define routes( $n$ ):  
2    $dp := \text{array}[0..n]$   
3    $dp[0] := 1$   
4    $dp[1] := 1$   
5    $dp[2] := 2$   
6   for  $i$  in  $[3..n]$ :  
7      $dp[i] := dp[i - 3] + dp[i - 2] + dp[i - 1]$   
8   return  $dp[n]$ 
```



Example: Increment or double

Starting at zero you can apply one of two operation:

- Increment by one
- Double

Minimum number of operations to get from 0 to n ?

n	0	1	2	3	4	5	6	7	8	9
ops	0	1	2	3	3	4	4	5	4	5

What is the base case(s)? What is the recurrence?

Example: Increment or Double

```
1 define inc_or_double( $n$ ):  
2    $dp := \text{array}[0..n]$   
3    $dp[0] := 0$   
4   for  $i \in [1..n]$ :  
5     if  $i$  is odd:  
6        $dp[i] := dp[i - 1] + 1$   
7     else:  
8        $dp[i] := \min(dp[\frac{i}{2}], dp[i - 1]) + 1$   
9   return  $dp[n]$ 
```

Complexity and space: $O(n)$

You are robbing houses on a street with n houses. The value of what you can steal in each house is $\{h_1, h_2, \dots, h_n\}$. You may not rob any houses that are adjacent to each other. How much can you get?

Robbing Houses

17/56

Define $\text{dp}[i]$ to be what you could steal if you only had $\{h_1, \dots, h_i\}$.

Base cases: $\text{dp}[0] = 0$, $\text{dp}[1] = h_1$

For any house h_i , you can either rob it or not.

If you rob h_i , you can't rob h_{i-1} .

$\text{dp}[i] := \max(h_i + \text{dp}[i - 2], \text{dp}[i - 1])$

$\text{dp}[n]$ is the answer.

Robbing Houses

```

1  dp := array of size  $n$ 
2  dp[0] := 0, dp[1] :=  $h[1]$ 
3  for  $i \in [2...n]$ :
4      | no_rob_v = dp[ $i - 1$ ]
5      | rob_v = dp[ $i - 2$ ] +  $h[i]$ 
6      | dp[ $i$ ] = max(no_rob_v, rob_v)
7  return dp[ $n$ ]

```

i	0	1	2	3	4	5	6	7	8	9	10
h		12	30	3	3	50	51	50	9	9	9
dp	0	12	30	30	33	80	84	130	130	139	139

And...?

Dynamic programs usually answer one question easily:

“What is the max that I can make on this street?”

Often there is a related question:

“Which houses should I rob to achieve that max?”

i	0	1	2	3	4	5	6	7	8	9	10
h		12	30	3	3	50	51	50	9	9	9
dp	0	12	30	30	33	80	84	130	130	139	139

Backtracking on dp

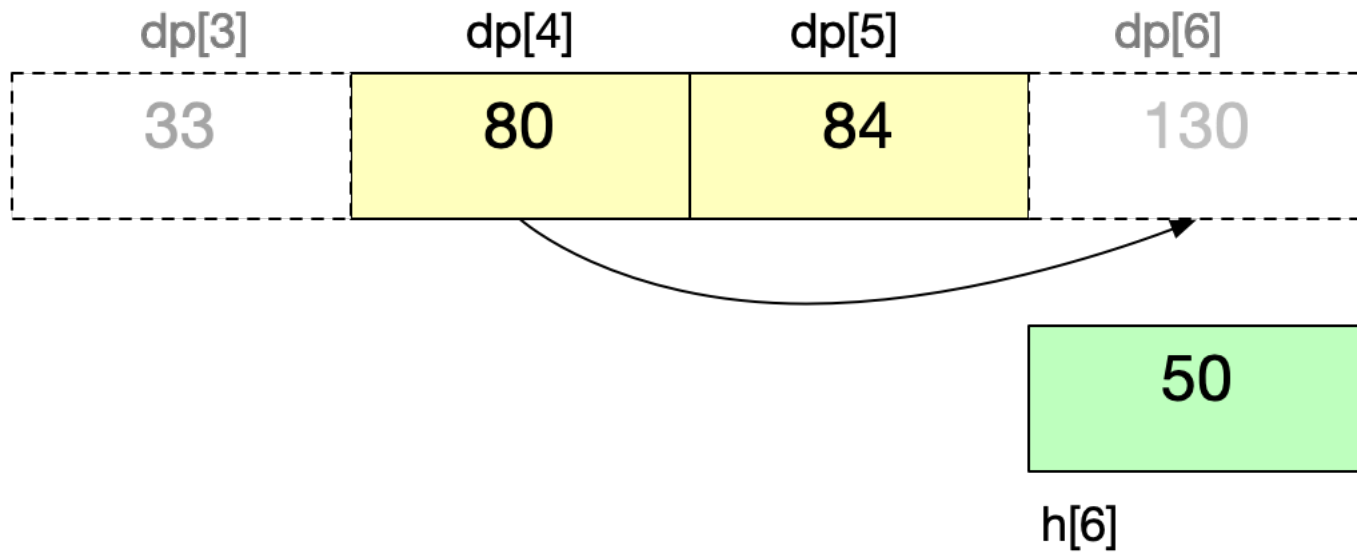
```

1 rob_set := an empty set
2  $i := n$ 
3 while  $i \geq 1$ :
4   if  $dp[i] == dp[i - 1]$ :
5      $i := i - 1$ 
6   else:
7     rob_set.add( $i$ )
8      $i := i - 2$ 
9 return rob_set

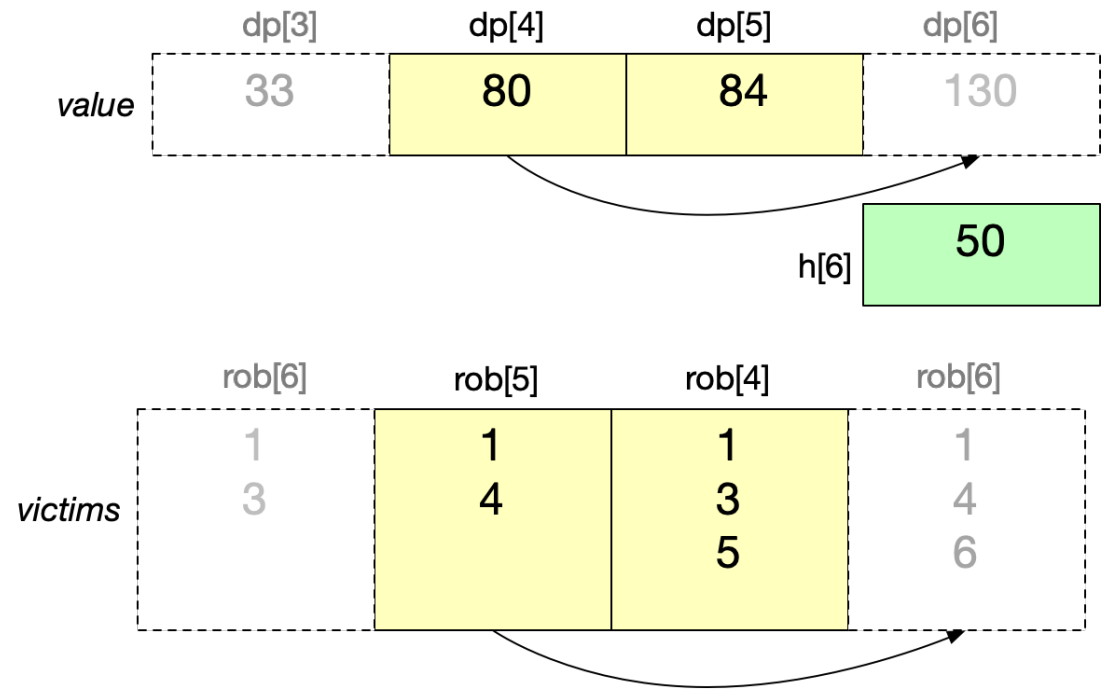
```

i	0	1	2	3	4	5	6	7	8	9	10
h		12	30	3	3	50	51	50	9	9	9
dp	0	12	30	30	33	80	84	130	130	139	139

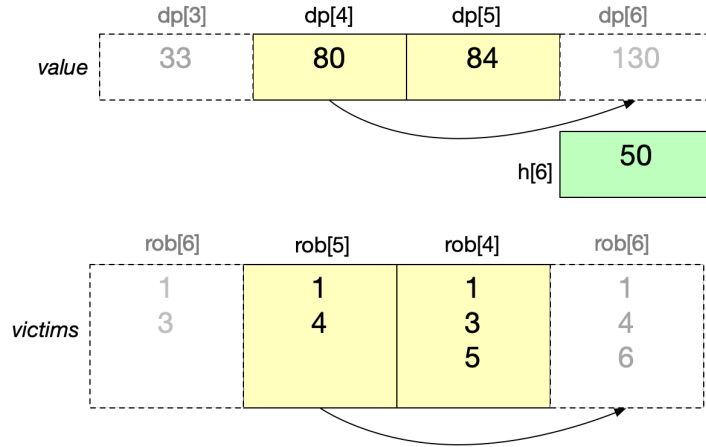
But what if you are tidy?



The idea: Gather as you go



Gathering as you go



```

1 dp := circular queue of theft values
2 rob_sets := circular queue of sets of houses to rob
3 dp.push_back(0), rob_sets.push_back({})
4 dp.push_back(h[1]), rob_sets.push_back({1})
5 for  $i \in [2..n]$ :
6      $v := dp.pop\_front() + h[i]$ 
7     house_set = rob_sets.pop_front()
8     if  $v > dp.peek\_front()$ :
9         dp.push_back( $v$ )
10        rob_sets.push_back(house_set  $\cup \{i\}$ )
11    else:
12        dp.push_back(dp.peek_front())
13        rob_sets.push_back(rob_sets.peek())
14 return (dp.back(), rob_sets.pop())
  
```

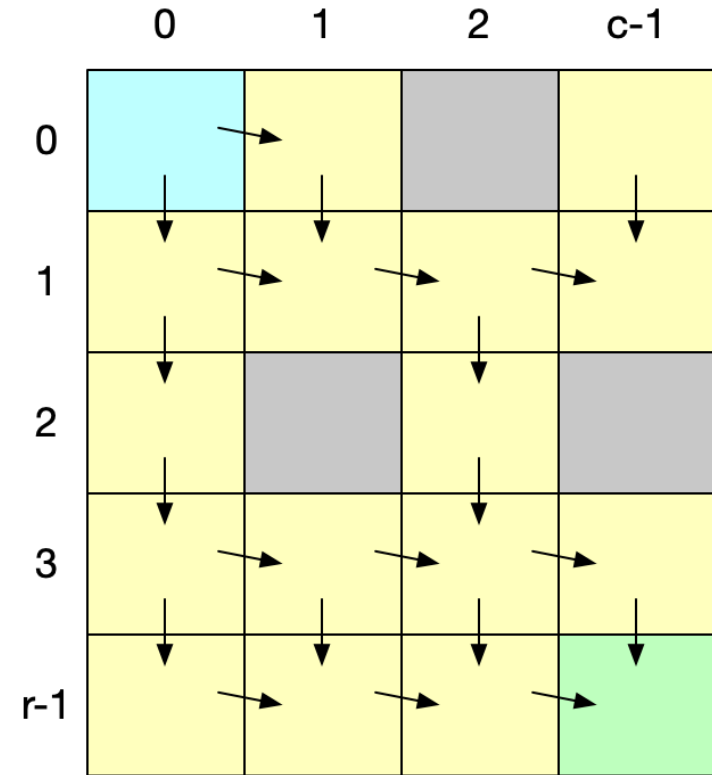
Example: Paths through a grid

You are given a $r \times c$ grid.

You can only move down or right.

Some of the squares may not be entered.

How many different routes from $(0,0)$ to $(r-1, c-1)$?



Use a 2D array for memoization

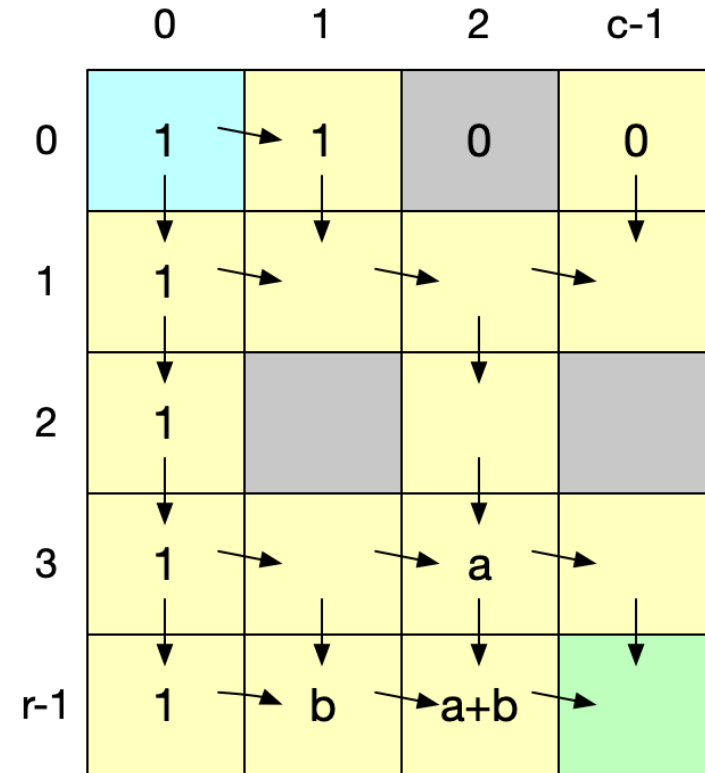
$dp[i, j]$ represents number of routes $(0,0)$ to (i,j) .

Base case:

- The cells on left are 1 if reachable, 0 if not
- The cells on the top are 1 if reachable, 0 if not

$$dp[i, j] = dp[i-1, j] + dp[i, j-1]$$

Answer is $dp[r-1, c-1]$



Implementation: Base case

```
dp := r by c array
```

```
dp[0,0] = 0
```

```
blocked := false;
```

```
for i in 1 to r-1:
```

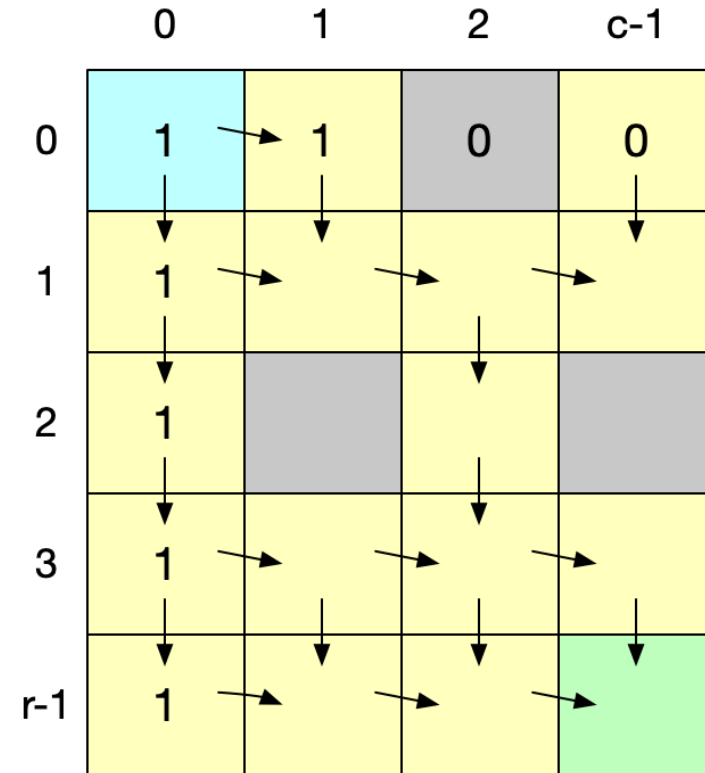
```
    if (i, 0) in forbidden:
```

```
        blocked := true;
```

```
    if blocked dp[i,0] = 0
```

```
    else db[i,0] = 1
```

...same with start of each column...



Implementation: Recurrence

```

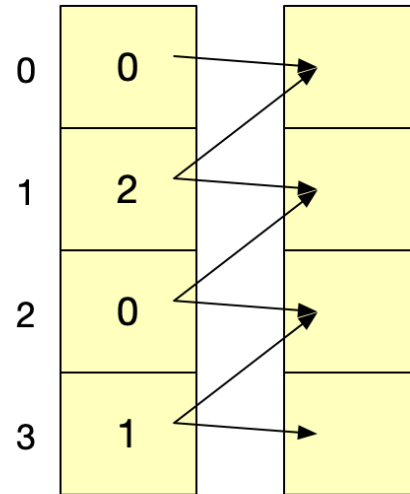
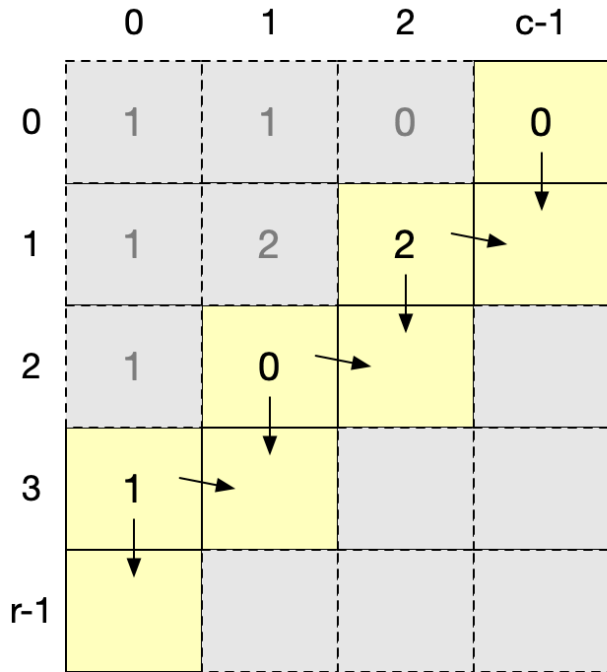
for i in 1 to r-1:
  for j in 1 to c-1:
    if (i, j) in forbidden:
      dp[i,j] = 0
    else:
      dp[i,j] = dp[i-1, j] + dp[i, j-1]
return dp[r-1, c-1]

```

	0	1	2	c-1
0	1	1	0	0
1	1	2	2	2
2	1	0	2	0
3	1	1	3	3
r-1	1	2	5	8

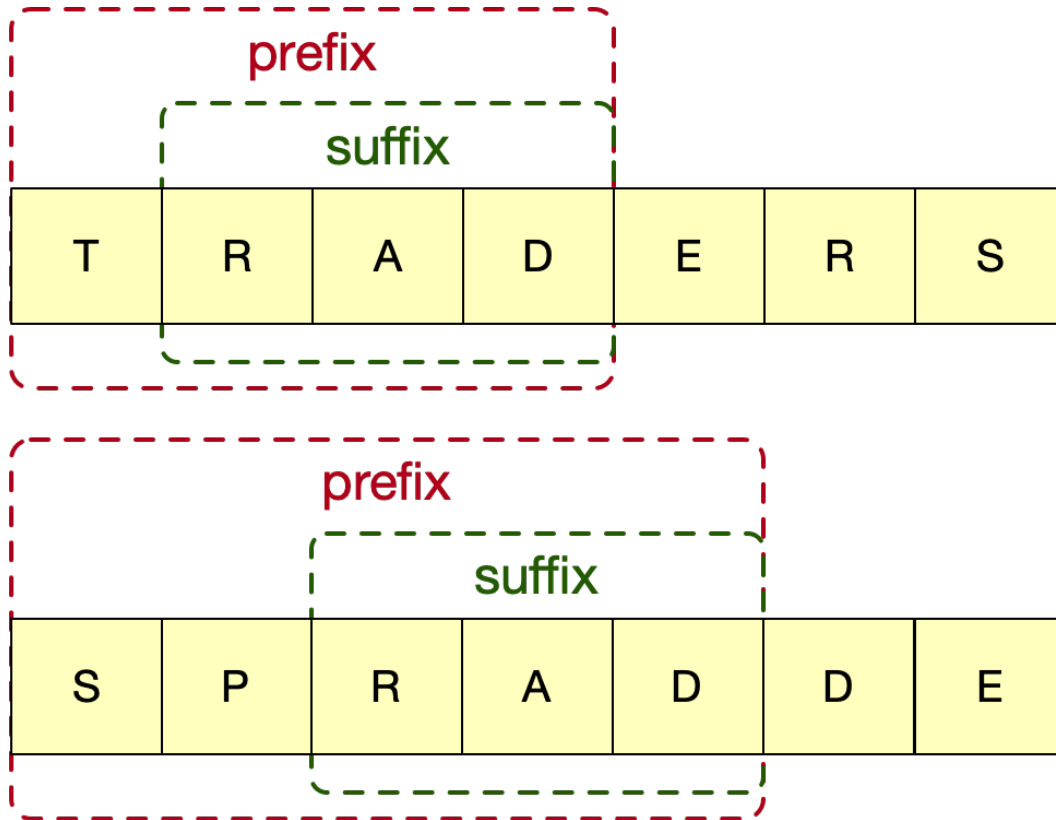
Tidy version

Keep swapping two 1-dimensional arrays



How long is the longest common
substring in “TRADERS” and
“SPRADDE”?

A Substring is a Suffix of a Prefix



Approach:

- Look at all possible prefixes for both strings
- Find length of longest common suffix

The bad recursive solution

Define `shortened()` to return the string missing its last character

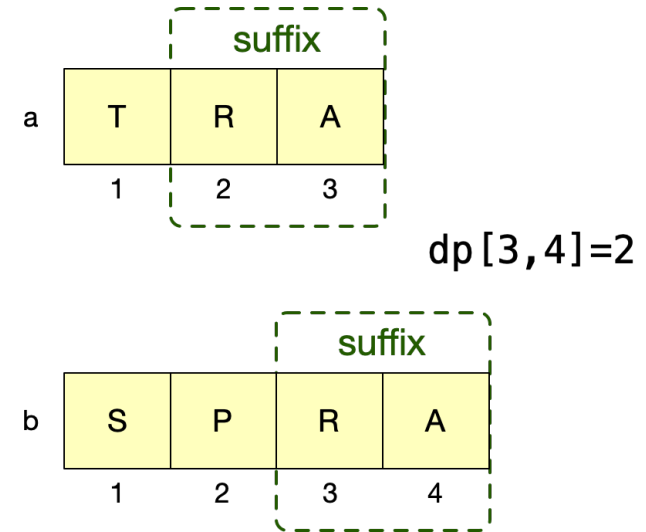
```
len_com_sub_str(a,b) -> int:  
    if a or b are empty strings, return 0  
  
    ender := len_of_common_suffix(a, b)  
    more_a := len_com_sub_str(a, b.shortened())  
    more_b := len_com_sub_str(a.shortened(), b)  
    return max(ender, more_a, more_b)
```

How many times is `len_com_sub_str` called for two seven-character strings? 6863 times!

Longest common substring

Given strings $[a_1, a_2, \dots, a_n]$ and $[b_1, b_2, \dots, b_m]$.

$dp[i, j]$ = length of common suffix of prefixes $[a_1, a_2, \dots, a_i]$ and $[b_1, b_2, \dots, b_j]$



	S	P	R	A	D	D	E
T							
R							
A				2			
D					3		
E						0	
R							
S							

Two strings have common suffix k letters long, you add a_i to one and b_j to the other:

- If $a_i == b_j$, how long is the common suffix now?
- If $a_i \neq b_j$, how long is the common suffix now?

```

if a[i] == b[j]:
    dp[i,j] := dp[i-1, j-1] + 1
else
    dp[i,j] := 0
  
```

Longest common substring: Base case?

		S	P	R	A	D	D	E
	0	0	0	0	0	0	0	0
T	0							
R	0							
A	0							
D	0							
E	0							
R	0							
S	0							

If one of the strings has zero length, how long is the common suffix?

Longest common substring

		S	P	R	A	D	D	E
	0	0	0	0	0	0	0	0
T	0	0	0	0	0	0	0	0
R	0	0	0	1	0	0	0	0
A	0	0	0	0	2	0	0	0
D	0	0	0	0	0	3	1	0
E	0	0	0	0	0	0	0	2
R	0	0	0	1	0	0	0	0
S	0	1	0	0	0	0	0	0

What is the length of the longest substring?

Longest common substring: pseudocode

		S	P	R	A	D	D	E
	0	0	0	0	0	0	0	0
T	0	0	0	0	0	0	0	0
R	0	0	0	1	0	0	0	0
A	0	0	0	0	2	0	0	0
D	0	0	0	0	0	3	1	0
E	0	0	0	0	0	0	0	2
R	0	0	0	1	0	0	0	0
S	0	1	0	0	0	0	0	0

```

dp := array (n+1) x (m+1) of zeros
for i in 1 to n:
  for j in 1 to m:
    if a[i] == b[j]:
      dp[i,j] := dp[i-1,j-1] + 1
return max(dp)

```

Yeah, but, *what* is the longest common substring?

Backtracking

		S	P	R	A	D	D	E
	0	0	0	0	0	0	0	0
T	0	0	0	0	0	0	0	0
R	0	0	0	1	0	0	0	0
A	0	0	0	0	2	0	0	0
D	0	0	0	0	0	3	1	0
E	0	0	0	0	0	0	0	2
R	0	0	0	1	0	0	0	0
S	0	1	0	0	0	0	0	0

Longest common substring: pseudocode

		S	P	R	A	D	D	E
	0	0	0	0	0	0	0	0
T	0	0	0	0	0	0	0	0
R	0	0	0	1	0	0	0	0
A	0	0	0	0	2	0	0	0
D	0	0	0	0	0	3	1	0
E	0	0	0	0	0	0	0	2
R	0	0	0	1	0	0	0	0
S	0	1	0	0	0	0	0	0

(Reminder: $a_1, \dots, a_n$ is one-based.)

`m := max(dp)`

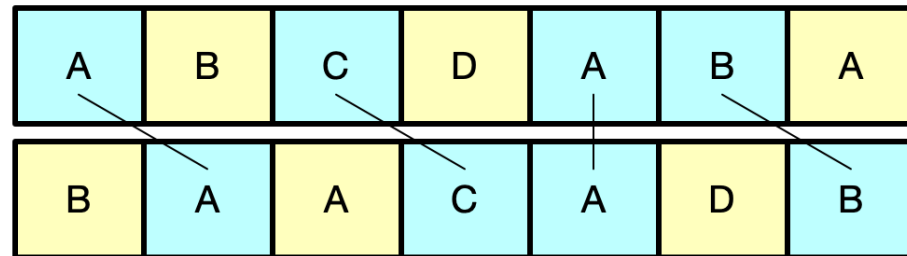
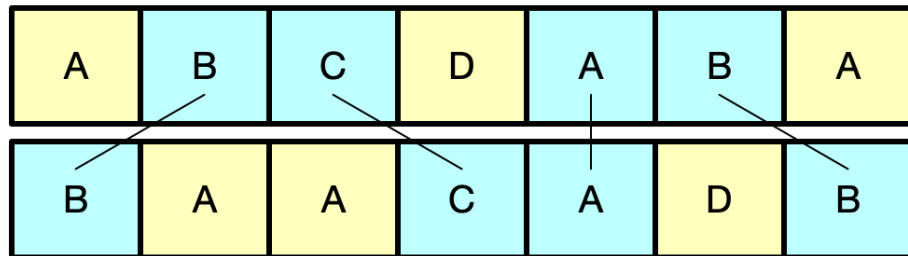
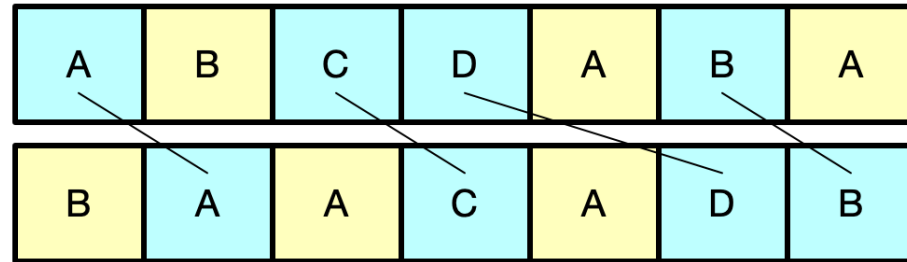
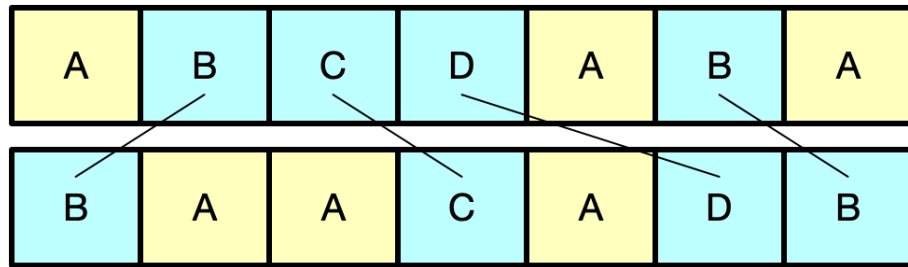
`(m_a, m_b) := argmax(dp)`

`s := m_a - m + 1`

`return substring(a, start=s, length=m)`

How long is the longest common
subsequence in “ABCDABA” and
“BAACADB”?

Length of longest common subsequence = 4



Longest common subsequence

Given sequences $[a_1, a_2, \dots, a_n]$ and $[b_1, b_2, \dots, b_m]$.

$dp[i, j]$ = length of max common subsequence of $[a_1, a_2, \dots, a_i]$ and $[b_1, b_2, \dots, b_j]$

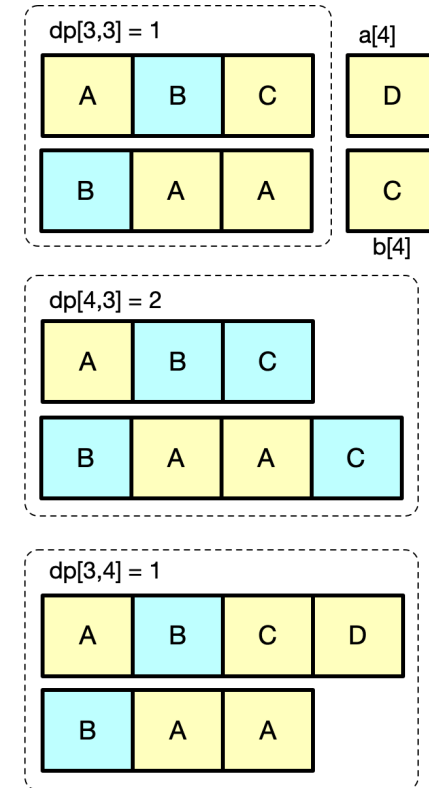
		B	A	A	C	A	D	B
	0	0	0	0	0	0	0	0
A	0	?	?	?	?	?	?	?
B	0	?	?	?	?	?	?	?
C	0	?	?	?	?	?	?	?
D	0	?	?	?	?	?	?	?
A	0	?	?	?	?	?	?	?
B	0	?	?	?	?	?	?	?
A	0	?	?	?	?	?	?	?

Recurrence: Longest common subsequence

$[a_1, \dots, a_{i-1}]$ and $[b_1, \dots, b_{j-1}]$ have a common subsequence of length k .

What if $a_i == b_j$? How long is the common subsequence of $[a_1, \dots, a_i]$ and $[b_1, \dots, b_j]$?

What if $a_i \neq b_j$?



Longest common subsequence: psuedocode

```

1  define longest_common_subsequence(a, b):
2       $n := \text{len}(a), m := \text{len}(b)$ 
3      for  $i \in [0...n]$  :  $\text{dp}[i, 0] := 0$ 
4      for  $j \in [0...m]$  :  $\text{dp}[0, j] := 0$ 
5      for  $i \in [1...n]$ :
6          for  $j \in [1...m]$ :
7              if  $a[i] == b[j]$ :
8                   $\text{dp}[i, j] := \text{dp}[i - 1, j - 1] + 1$ 
9              else:
10                  $\text{dp}[i, j] := \max(\text{dp}[i - 1, j], \text{dp}[i, j - 1])$ 
11  return  $\text{dp}[n, m]$ 

```

		B	A	A	C	A	D	B
	0	0	0	0	0	0	0	0
A	0	0	1	1	1	1	1	1
B	0	1	1	1	1	1	1	2
C	0	1	1	1	2	2	2	2
D	0	1	1	1	2	2	3	3
A	0	1	2	2	2	3	3	3
B	0	1	2	2	2	3	3	4
A	0	1	2	3	2	3	3	4

Yeah, but *what* are the longest
common subsequences in “ABCDABA”
and “BAACADB”?

dp array as a DAG

		B	A	A	C	A	D	B
	0	0	0	0	0	0	0	0
A	0	0	1	1	1	1	1	1
B	0	1	1	1	1	1	1	2
C	0	1	1	1	2	2	2	2
D	0	1	1	1	2	2	3	3
A	0	1	2	2	2	3	3	3
B	0	1	2	2	2	3	3	4
A	0	1	2	3	2	3	3	4

Every path represents a common subsequence.

Paths that starts at a 1 and end at 4 is a *maximal* common subsequence.

Maximal Common Subsequences

		B	A	A	C	A	D	B
	0	0	0	0	0	0	0	0
A	0	0	1	1	1	1	1	1
B	0	1	1	1	1	1	1	2
C	0	1	1	1	2	2	2	2
D	0	1	1	1	2	2	3	3
A	0	1	2	2	2	3	3	3
B	0	1	2	2	2	3	3	4
A	0	1	2	3	2	3	3	4

BCDB

ACDB

BCAB

ACAB

		B	A	A	C	A	D	B
	0	0	0	0	0	0	0	0
A	0	0	1	1	1	1	1	1
B	0	1	1	1	1	1	1	2
C	0	1	1	1	2	2	2	2
D	0	1	1	1	2	2	3	3
A	0	1	2	2	2	3	3	3
B	0	1	2	2	2	3	3	4
A	0	1	2	3	2	3	3	4

```

1  define sequences_ending_at( $i, j, a, b, dp$ ):
2      if  $a[i] == b[j]$ :
3          if  $dp[i][j] == 1$ :
4              return new set  $\{(a[i])\}$ 
5          else:
6               $r := \text{sequences\_ending\_at}(i - 1, j - 1, a, b, dp)$ 
7              Append  $a[i]$  to each tuple in  $r$ 
8              return  $r$ 
9      else:
10          $r := \text{empty set}$ 
11         if  $dp[i - 1][j] == dp[i][j]$ :
12              $r := \text{union}(r, \text{sequences\_ending\_at}(i - 1, j, a, b, dp))$ 
13         if  $dp[i][j - 1] == dp[i][j]$ :
14              $r := \text{union}(r, \text{sequences\_ending\_at}(i, j - 1, a, b, dp))$ 
15         return  $r$ 

```


How long is the longest palindromic
subsequence of ABDCBBA?

'ABCBA' or 'ABBBA'

Already done longest common subsequences?

Longest pandomic subseq of $[a_1, \dots, a_n]$ is longest common subsequence of $[a_1, \dots, a_n]$ and $[a_n, \dots, a_1]$

$dp[i, j]$ will be length longest pandromic subsequence of $[a_i, \dots, a_j]$ for $i \leq j$

Base cases:

- $i = j$: $dp[i, j] = 1$
- $i > j$: $dp[i, j] = 0$

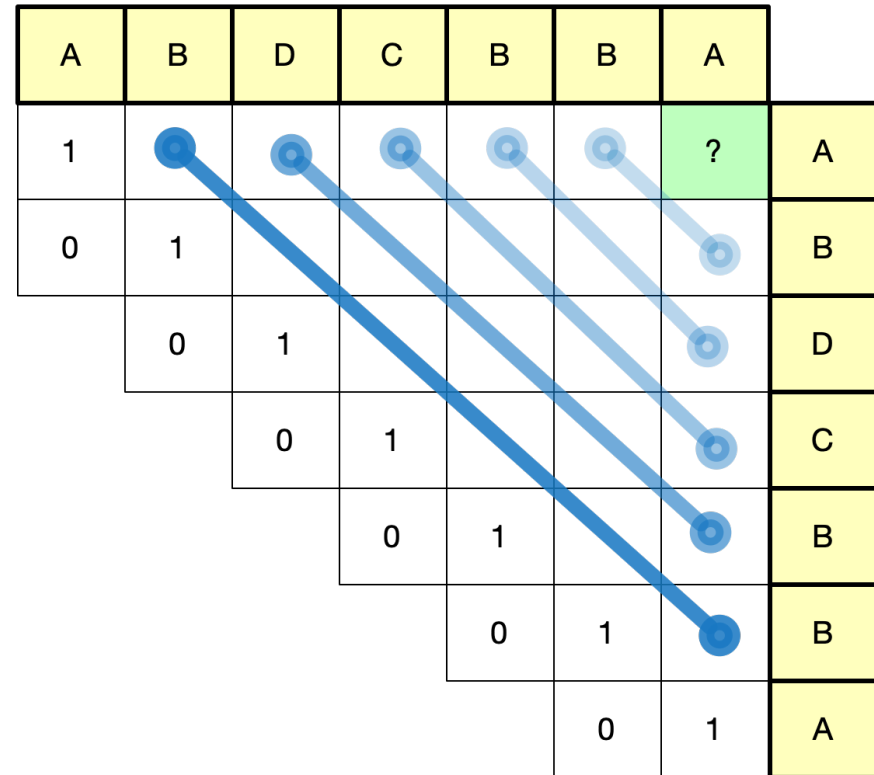
Answer: $dp[1, n]$

last letter in substring							first letter in substring
A	B	D	C	B	B	A	
1						?	
0	1						
	0	1					
		0	1				
			0	1			
				0	1		A
					0	1	

Recurrence direction?

```

for s in 1..n:
  for r in 1..n-s:
    c = r + s
    dp[r,c] = ?
  
```



Recurrence

if $a_i = a_j$, a letter is added to both sides of the palindrome of $a_{i+1} \dots a_{j-1}$.

else, use the max of $a_{i+1} \dots a_j$ and $a_i \dots a_{j-1}$

last letter in substring							
A	B	D	C	B	B	A	
1						?	A
0	1						B
	0	1					D
		0	1				C
			0	1			B
				0	1		B
					0	1	A
							first letter in substring

```
1 dp :=  $n \times n$  array of zeroes
2 for  $i \in [1..n]$ :
3   dp[i, i] := 1
4   for  $s \in [1..n]$ :
5     for  $i \in [1..n - s]$ :
6        $j := i + s$ 
7       if  $a[i] = a[j]$ :
8         | dp[i, j] := dp[i + 1, j - 1] + 2
9       else:
10        | dp[i, j] := max(dp[i + 1, j], dp[i, j - 1])
11 return dp[1, n]
```

A	B	D	C	B	B	A	
1	1	1	1	3	3	5	A
0	1	1	1	3	3	3	B
	0	1	1	1	2	2	D
		0	1	1	2	2	C
			0	1	2	2	B
				0	1	1	B
					0	1	A

Questions?

Slides by Aaron Hillegass